Documentation and Quality Gates

Module 3 — Onboarding & Industrialize the Codebase

Instructor **Angelo Sotgiu** · with **Battista Biggio**

Industrial Software Development M.Sc. in Computer Engineering, Cybersecurity and Artificial Intelligence University of Cagliari, Italy

Code Quality and Refactoring and *Modularity and Python Modules* fixed the code by hand. Today we install the tools that check code **automatically**, and the documentation that is generated from it.

Concepts covered: **docstrings** and a documentation site generated from them (NumPy style, Sphinx), **formatting and linting** with ruff (including its security rules), the **software supply chain**, **static typing** with mypy, and **pre-commit**, which runs all checks before every commit.

Documentation

Docstrings

A **docstring** is the string literal right under a `def`, `class` or at the top of a module. It states the **contract**: what the code does, its inputs, its output, its errors. `help()` prints it, IDEs show it, and documentation tools read it.

PEP 257 sets the basic rules: triple double quotes; a one-line summary, written as a command ("Return…", not "Returns…"); then, if needed, a blank line and the details. Two docstrings of `heron`, from `parser.py` (the module and one function):

```
"""Read the triage-relevant parts of a raw ``.eml`` message."""
...
def _domain(header_value: str) -> str | None:
    """Return the domain of the address in a From/Reply-To value, or None."""
```

Comments say **why** the code is written this way; docstrings say **what** it promises to its callers.

Spot the violation

Two example docstrings we may have written in `heron`. Which PEP 257 rule do they break?

```python
def report(results: list[TriageResult]) -> None:
    """A line for each email, with its verdict and its score."""
```

```python
def auth_score(email: ParsedEmail) -> int:
    """Points when SPF or DMARC failed."""
```

They describe the result instead of giving a command. As a command: "Print one line for each email, with its verdict and its score."

A **linter** (e.g., ruff) can automatically check this rule.

Docstring styles

For longer docstrings, a team picks one **style** for the details and uses it everywhere. Three styles are common: reStructuredText fields (`:param raw:`), the **NumPy style** and the Google style (`Args:`). The tools read all three; consistency matters more than the choice.

This course uses the **NumPy style** (the *numpydoc* standard). NumPy, SciPy and scikit-learn use it too. After the summary line and a blank line, each section has a title underlined with dashes:

- `Parameters`: one entry per parameter, `name : type` on one line, the description below it, indented by four spaces
- `Returns`: the type of the result, then its description
- other sections when they are needed: `Raises` (the exceptions the function raises on purpose), `Examples`
- double backticks mark code, as in ``` ``.eml`` ``` above (reStructuredText markup, see step 2)

Let the IDE write the first draft

Typing the sections by hand is slow. Two tools write a first draft for you:

- **PyCharm** knows the NumPy layout. Set it once: *Settings → Tools → Python Integrated Tools → Docstring format: NumPy*. Then type `"""` on the line under a `def` and press Enter: PyCharm writes the summary line and the `Parameters` and `Returns` sections, with the names and the types of the signature
- **GitHub Copilot** (the *GitHub Copilot* plugin of PyCharm, signed in with your GitHub account) proposes the descriptions as grey text while you type; `Tab` accepts a proposal. The Student plan of GitHub Education gives unlimited code completions (as of October 2026)

The draft is **yours to check**: the tool guesses from the names and the code. Read every line: does it say what the function promises to its callers? Are the types the ones of the signature? Is the summary line a command? A docstring that lies is worse than no docstring.

Step 1: NumPy-style docstrings

The **public** functions, the ones other modules call, get the full contract. In `scorer.py`, `triage()`:

```
def triage(email: ParsedEmail) -> TriageResult:
    """Run every check on one message and decide its verdict.

    Parameters
    ----------
    email : ParsedEmail
        The message, as returned by ``parse_eml``.

    Returns
    -------
    TriageResult
        The score of the message and its verdict.
    """
```

Do the same for `parse_eml()` in `parser.py` (two parameters), with the PyCharm stub and Copilot; the appendix shows mine, yours are in your own words. In `cli.py`, make the docstring of `report()` a command: "Print one line for each email, with its verdict and its score." A short helper such as `_domain()` keeps its one-line docstring.

Step 1: check and commit

Python keeps each docstring in the attribute `__doc__` of the function, which is what `help()` prints. Check that yours is there, then commit:

```
uv run python -c "import heron.scorer; print(heron.scorer.triage.__doc__)"
git add src
git commit -m "docs: NumPy-style docstrings for the public functions"
```

The types are written twice: in the signature, where mypy checks them, and in the docstring, for the reader. Keep them the same when the code changes. A tool can check part of the rules automatically: step 3 introduces the linter ruff, which has rules for docstrings too.

Step 2: a documentation site with Sphinx

A **documentation generator** turns hand-written pages **and** the docstrings into a website. `heron` uses **Sphinx**: its extension `autodoc` imports the modules and reads their docstrings, and `napoleon` understands the NumPy sections. A Sphinx project needs two short files that we write once, and the pages of the API reference, which a tool writes for us. First, create the folder `docs/` and the settings file `docs/conf.py`:

```
"""Sphinx configuration: build the heron docs from the docstrings."""

project = "heron"
extensions = [
    "sphinx.ext.autodoc",   # read the docstrings of the modules
    "sphinx.ext.napoleon",  # understand the NumPy-style sections
]
napoleon_google_docstring = False  # this project writes NumPy style only
html_theme = "sphinx_rtd_theme"
exclude_patterns = ["_build"]
```

- `html_theme`: the Read the Docs theme (the package `sphinx-rtd-theme`, installed in step 2 of *Environments, Packaging and Design Patterns*); `exclude_patterns`: the site is built into `docs/_build/`, which Sphinx must not read as a source
- no `sys.path` line: `uv sync` already installed `heron` in `.venv`, so `import heron` works for Sphinx as for any other program run with `uv run`

Step 2: the home page

Sphinx pages are written in **reStructuredText** (`.rst`), the markup language of the Python documentation. A title is underlined with `=`; a **directive**, `.. name::`, inserts special content.

`docs/index.rst` is the home page, written by hand: in your own words, what `heron` does and the two commands that run it. The `toctree` directive (table of contents) links the other pages, by file name without `.rst`. `modules` is the first page of the API reference, which `sphinx-apidoc` writes on the next slide:

```
heron
=====

Phishing email triage: heron reads ``.eml`` files and gives each message a
verdict (benign, suspicious or phishing) and a score.

.. code-block:: bash

   uv sync                       # install heron and the dev tools from uv.lock
   uv run heron data/samples     # one verdict per message

.. toctree::

   modules
```

Step 2: generate the API pages

We do not write the pages of the API reference by hand. **`sphinx-apidoc`**, a command that comes with Sphinx, reads the package and writes them:

```
uv run sphinx-apidoc -o docs src/heron   # writes docs/modules.rst and docs/heron.rst
```

`docs/heron.rst` has one section for each module of the package, with an `automodule` directive. When Sphinx builds the site, `autodoc` replaces each directive with the documentation of the module, built from its docstrings:

```
heron.cli module
----------------

.. automodule:: heron.cli
   :members:
   :show-inheritance:
   :undoc-members:
```

The generated pages are source files: commit them with the rest. When the package gets a new module, run the command again with `-f` (overwrite the pages): `uv run sphinx-apidoc -f -o docs src/heron`.

Step 2: build the site

```
uv run sphinx-build -W docs docs/_build   # -W: any warning is an error
# open docs/_build/index.html in your browser
git add docs
git commit -m "docs: generate an API reference with Sphinx"
```

- open the page *heron package*: every module, class and function, with its signature and its docstring; the NumPy sections of `triage()` become a list of parameters and a return type
- a function without a docstring appears with no description (`:undoc-members:` lists it anyway): an easy way to find missing documentation
- `docs/_build/` is an output, not source: the Python `.gitignore` of GitHub (step 0 of *Code Quality and Refactoring*) already excludes it

MkDocs with the plugin *mkdocstrings* is the other common generator; its pages are Markdown. Whatever the tool, the documentation lives in the same repository as the code. In *Continuous Integration and Delivery*, CI rebuilds it with `-W` on every change, so broken markup or a module that no longer imports stops the build. Sphinx does not check that a `Parameters` section still matches the signature: that needs a docstring checker, such as pydoclint.

README and CHANGELOG

Next to the generated reference, most projects keep two hand-written files:

- **README**: what the project is, how to install it, how to run it. It is the first page anyone reads. We'll update `heron`'s README later
- **CHANGELOG**: what changed in each version, written for users, so they know what an upgrade brings. It pairs with semver; a common format is Keep a Changelog. `heron` keeps no CHANGELOG file: later in the course, each release gets notes that GitHub writes from the merged pull requests

Each document has its own reader: the README is for the users who install and run the project, the API reference for the programmers who call its code, `CONTRIBUTING.md` for the people who change it, and the architecture decision records explain to future maintainers why the code is built the way it is.

Quality gates

Move quality left

The later a defect is found, the more it costs: at review it is a comment, in production it is an incident. A widely quoted rule of thumb (usually attributed to Barry Boehm) says the cost grows 10–100× from early to late stages; the exact numbers are debated, but the direction is well supported. A **quality gate** is a check that a change must pass before it moves to the next stage. We run every gate **as early as possible**:



Mechanical problems (layout, unused names, wrong types) are caught by tools at commit time. Human review then spends its attention on design and intent, where tools cannot help.

Linters and formatters

A **linter** is a program that reads the source code **without running it** (*static analysis*) and reports every place that breaks one of its rules. Each finding names the file, the line, a rule **code** and a short message, for example `F841 Local variable 'subj' is assigned to but never used`. The rules come in families:

- **likely bugs**: an undefined or unused name, a mutable default argument, a bare `except`
- **style**: the layout rules of PEP 8, the order of the imports, the docstring rules of PEP 257
- **simpler or more modern code**: a loop that can be a comprehension, old syntax with a newer form
- **security**: patterns that are often vulnerabilities

The team chooses which rules apply, once, in a configuration file; the linter then checks every file in the same way, in seconds. A **formatter** is a different tool: it reports nothing, it **rewrites** the layout of the code in one standard way.

ruff

ruff (by Astral, the makers of `uv`) is a linter **and** a formatter in one program, written in Rust: it checks a whole project in a fraction of a second. It re-implements the rules of many older Python tools and keeps their codes, with a letter prefix per tool: `F` (pyflakes), `E` (pycodestyle, PEP 8), `I` (isort, imports), `B` (bugbear), `S` (bandit, security), `D` (pydocstyle, docstrings) and many more.

```
uv run ruff check .                    # lint: report the findings of the selected rules
uv run ruff check --select D401 src    # one run with one rule only (D401: summary line as a command)
uv run ruff rule F841                  # explain a rule: why it exists, an example, the fix
uv run ruff check --fix .              # apply the fixes that ruff knows are safe
uv run ruff format .                   # format: rewrite the layout
```

Without configuration, ruff checks only a small default set of `E` and `F` rules. The configuration lives in `pyproject.toml`, in the tables `[tool.ruff]` and `[tool.ruff.lint]`: step 3 writes it. To accept one finding on purpose, write `# noqa: <code>` at the end of its line, with the reason.

Step 3: formatting with ruff

`ruff format` produces the same layout as the older formatter `black`, so nobody argues about style in reviews. Before any configuration, ask which files it would change:

```
uv run ruff format --check .
```

ruff uses a default maximum line length of 88 characters. The line length is a **team decision**, written once in `pyproject.toml`. Let's set it to 100 characters:

```
[tool.ruff]
line-length = 100
src = ["src"]
```

```
uv run ruff format --check .
```

ruff also formats the code blocks inside Markdown files, so it counts the `.md` files too.

Step 3: linting, to find likely bugs

Now choose the linter's rules. Add the rule sets to the same `pyproject.toml` (one commit with the formatter settings):

```toml
[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B", "SIM", "C4", "S"]  # S = security rules (bandit)

[tool.ruff.lint.per-file-ignores]
"tests/*" = ["E501", "S101"]  # pytest uses plain assert
```

- `E` style errors (pycodestyle), `F` unused or undefined names (pyflakes), `I` import order
- `UP` old syntax that has a modern form, `B` bug-prone patterns (bugbear), `SIM`/`C4` code that can be simpler
- `S` security rules (from the tool bandit)

```
uv run ruff check .           # on my code: "All checks passed!"
git add pyproject.toml
git commit -m "chore: configure ruff (format and lint)"
```

Your code may differ from mine (your docstrings, for example), so ruff may report something on it. Read each finding, fix it by hand, and commit the fixes on their own.

Step 4: let the formatter write

Now let ruff rewrite the files that may need changes. You type nothing:

```
uv run ruff format .
git diff                        # read what the tool changed before you commit it
git add src
git commit -m "style: format the code with ruff"
```

As an example, here `features.py` changes in two places. The `KEYWORDS` list does not fit in 100 characters, so ruff writes one item per line; the `sum()` in `brand_score()` fits in 100 characters, so ruff joins its two lines into one:

```
-KEYWORDS = ["urgent", "verify", "suspended", "password", "expires", "act now",
-            "congratulations", "winner", "claim", "immediately", "gift card"]
+KEYWORDS = [
+    "urgent",
+    "verify",
+    ...
+    "gift card",
+]
...
-    return sum(W_BRAND_MISMATCH for brand in BRANDS
-               if brand in low and brand + ".com" not in low)
+    return sum(W_BRAND_MISMATCH for brand in BRANDS if brand in low and brand + ".com" not in low)
```

Commit configuration, tool output and hand-written fixes **separately**: each diff stays small, and a reviewer knows that a `style:` commit changes only the layout.

What the linter finds in the old script

Run the same rules on the initial `triage.py`, copied into a scratch file:

```
git show $(git rev-list --max-parents=0 HEAD):triage.py > old_triage.py  # the first commit; never commit this file
uv run ruff check old_triage.py --output-format concise
# old_triage.py:3:1:   I001 Import block is un-sorted or un-formatted
# old_triage.py:13:61: B006 Do not use mutable data structures for argument defaults
# old_triage.py:18:15: SIM115 Use a context manager for opening files
# old_triage.py:22:9:  E722 Do not use bare `except`
# old_triage.py:27:13: F841 Local variable `subj` is assigned to but never used
# old_triage.py:55:9:  S110 `try`-`except`-`pass` detected, consider logging the exception
# ... 9 findings in total (E722 three times, SIM115 twice)
```

The tool finds the mutable default, the files without `with`, the bare `except`, the dead variable: four smells of *Code Quality and Refactoring*, in one second. It does **not** find the hard-coded path, the magic numbers, the copied brand checks, the long function or the regex parser: those need a human reader. Keep `old_triage.py` for now: the type checker reads it too.

The security rules (S)

The `S` rules flag code patterns that are often **vulnerabilities**. Let's try on this code snippet:

```python
import sqlite3
import urllib.request

API_TOKEN = "sk-live-1234"  # a secret in the source code


def fetch(url: str) -> bytes:
    with urllib.request.urlopen(url) as response:  # any scheme, incl. file://
        return bytes(response.read())


def count(db: sqlite3.Connection, verdict: str) -> int:
    sql = f"SELECT COUNT(*) FROM verdicts WHERE verdict = '{verdict}'"  # SQL injection
    return int(db.execute(sql).fetchone()[0])
```

Save it as `risky_example.py` and run only the `S` rules on it:

```
uv run ruff check --select S --output-format concise risky_example.py
```

A finding is a **question**, not a verdict. If the code is safe, say why in a comment that a reviewer can check: `# noqa: S310 - <reason>`.

Reading lint findings

`ruff check --fix` applies the automatic fixes. Do not run it blindly:

- read the rule code and why it fired: `ruff rule B006` prints the explanation
- a `B006` mutable default or an `E722` bare `except` is a real latent bug, not noise: understand it before you change it
- then use `--fix` for the safe automatic fixes, and review the diff

As in step 4, commit the lint fixes on their own, separate from configuration and formatting.

The supply chain

What you trusted with uv add

Earlier in the course, `uv add --dev` installed five tools, which brought in 38 more packages (in the reference lockfile). Each one runs on your machine with your permissions. Together they are your **software supply chain**, and attackers target it (OWASP Top 10 2025, A03):

- **known vulnerabilities** in an old version you never updated
- **typosquatting**: a package with a look-alike name (`python-dateutil` vs `python-datutil`) that contains malware
- **slopsquatting**: an AI assistant invents a plausible package name; an attacker registers that name with malware; the next install runs it
- **dependency confusion**: your company has an internal package `acme-utils`; an attacker publishes `acme-utils` with a higher version on the public index, and an installer that searches both picks the public one
- **a compromised maintainer**: a real, popular package gets a malicious release (a famous example here)

Defending the supply chain

Each attack has a control, and you already use the first ones:

- **the lockfile**: `uv.lock` records exact versions *and hashes*; a dependency changes only through a reviewed diff, and a file with a different hash is refused
- **one index per package**, against dependency confusion: `uv` takes a package from the *first* index that has it and does not mix versions across indexes (`pip` with an extra index does)
- **check every new name** before you add it: does the package exist, is it the well-known one, is it maintained? Even more when an AI assistant suggested it
- **scanning** for known vulnerabilities (`pip-audit`) and a **software bill of materials** come with the CI security job

`heron` keeps the surface small: its runtime needs only the standard library, and only developer tools are installed.

Static typing

Type hints and a type checker

Type hints (`score: int`, `-> Verdict`) say what kind of value a variable, parameter or return value holds. Python ignores them at run time. A **type checker** such as **mypy** reads them and reports, without running the code, every place where the types do not fit. In `scorer.py`:

```python
def verdict_for(score: int) -> Verdict:
    """Map a score to a verdict."""
    if score >= PHISHING_THRESHOLD:
        return Verdict.PHISHING
    if score >= SUSPICIOUS_THRESHOLD:
        return Verdict.SUSPICIOUS
    return Verdict.BENIGN
```

A call `verdict_for("4")` is reported before the program runs: *Argument 1 to "verdict_for" has incompatible type "str"; expected "int"*.

Step 5: mypy in strict mode

Add the configuration of mypy for the whole package to `pyproject.toml`.:

```toml
[tool.mypy]
strict = true
files = ["src"]
```

```
uv run mypy src                # -> Success: no issues found in 7 source files
git add pyproject.toml
git commit -m "chore: type-check the package with mypy in strict mode"
```

Strict mode requires a type on every function, and reports the common ways in which a value silently gets the type `Any` (the type that switches checking off), such as a call to a function without annotations. `heron`'s code was written with types from the start, so mypy passes at once. On code you write, when mypy complains, **add the missing annotation** instead of silencing it with `# type: ignore`.

Optional values: `X | None`

`str | None` means "a string, or `None`". The most common real bug is to forget the `None` case. mypy on the initial `triage.py`, the scratch file of the linter slide:

```
uv run mypy old_triage.py
# old_triage.py:21: error: Item "None" of "Match[str] | None" has no attribute "group"  [union-attr]
# ... the same error 5 times, plus 5 errors for missing annotations
rm old_triage.py              # the scratch file is no longer needed
```

`re.search` returns a `Match`, or `None` when nothing matches. `triage.py` called `.group()` directly and hid the crash with a bare `except:`. mypy points at the bug, without any test.

Narrowing

In `heron`, `ParsedEmail.from_domain` has the type `str | None`: a message whose From header has no address has no domain. So mypy refuses any line that works only for a string, because the value may be `None`:

```
domain = email.from_domain.lower()
error: Item "None" of "str | None" has no attribute "lower"  [union-attr]
```

The fix is a **check for `None`** before the value is used. After the check, mypy knows that the value is a `str`: the type has become smaller. This is called **narrowing**. The common forms:

- `if x is None: return ...` (an early return): below this line, `x` is a `str`
- `if x:` or `if x is not None:`: inside the `if`, `x` is a `str`
- `isinstance(x, str)`: inside the `if`, `x` is a `str`

Narrowing in heron

`reply_to_score()` compares two values of type `str | None`:

```
def reply_to_score(email: ParsedEmail) -> int:
    """Points when replies go to a different domain than the sender's."""
    from_domain, reply_domain = email.from_domain, email.reply_to_domain
    if from_domain and reply_domain and from_domain != reply_domain:
        return W_REPLY_TO_MISMATCH
    return 0
```

The `if` narrows both values: the comparison runs only when both are real domains. Without the first two conditions, a message with no Reply-To header would compare its domain with `None`, find them different, and get 2 points by mistake.

The type `str | None` forces a decision that is easy to forget: what happens when there is no value? mypy reports every place where the code has not decided.

Generic types and Any

A **generic** type has parameters in square brackets: `list[str]`, `dict[str, int]`, `tuple[str, ...]` (a tuple of any length, all `str`, such as `ParsedEmail.links`). The type of `ALL_CHECKS` says exactly what a check is:

```
ALL_CHECKS: tuple[Callable[[ParsedEmail], int], ...] = (
    keyword_score,
    link_score,
    brand_score,
    auth_score,
    reply_to_score,
)
```

"A tuple of any length; each element is a function that takes one `ParsedEmail` and returns an `int`." mypy checks every function added to the tuple against this type.

`Any` switches the checker off for a value: everything is allowed on it, so mistakes pass unnoticed and crash at run time. Strict mode reports the common sources of *implicit* `Any`: functions without annotations, generic types without parameters (a bare `list`), and an `Any` returned from a typed function. Data from outside (JSON, HTTP requests) must be **validated** into real types instead.

Other type checkers

`mypy` is the reference type checker; `pyright` (behind VS Code's Pylance) is the other established one. Two new checkers, written in Rust, are often 10× faster or more on large code bases:

- **pyrefly** (Meta): stable 1.0 since May 2026
- **ty** (Astral, the makers of ruff and uv): still in **beta** as of October 2026

They read the same type hints, so switching does not change your code. `heron` stays on mypy strict: it is mature and its errors are well documented. Re-check this choice every year: the field moves fast.

The commit gate

Git hooks

A check that people must remember to run is soon forgotten. Git can run it for them: a **hook** is a script that git runs by itself at a fixed moment. The hooks of a clone live in its folder `.git/hooks/`, one executable file per moment, named after it:

- `pre-commit`: runs before git creates a commit. If the script ends with a **non-zero exit code**, git **cancels** the commit
- `commit-msg`: runs on the message you wrote, for example to check that it follows conventional commits
- `pre-push`: runs before `git push` sends anything

A new clone has only examples there (`pre-commit.sample` and others), which do nothing.

The pre-commit framework

Writing hooks by hand has two problems:

- `.git/` is never committed, so a hook is **not shared**: every clone of the team starts without it
- a hook that runs ruff, mypy and file checks is a long shell script, and each tool must be installed where the script expects it

pre-commit is a tool (a Python package) that solves both:

- the list of checks is a file of the repository, `.pre-commit-config.yaml`, committed and reviewed like any other file
- `pre-commit install` writes **one** small script, `.git/hooks/pre-commit`, which starts pre-commit before each commit
- pre-commit downloads each check at the exact version the file names, and installs it in its own environment

Step 6: pre-commit

Create `.pre-commit-config.yaml` at the root of the repository (versions as of October 2026):

```yaml
repos:
  - repo: https://github.com/astral-sh/ruff-pre-commit
    rev: v0.16.10
    hooks:
      - id: ruff-check
        args: [--fix]
      - id: ruff-format
  - repo: local
    hooks:
      - id: mypy
        name: mypy
        entry: uv run --locked mypy  # the mypy of the project's .venv, with every dependency
        language: unsupported  # pre-commit builds no environment: the project has one
        files: ^src/
        pass_filenames: false  # mypy checks files = ["src"] of pyproject.toml
  - repo: https://github.com/pre-commit/pre-commit-hooks
    rev: v6.0.0
    hooks:
      - id: end-of-file-fixer
      - id: trailing-whitespace
      - id: check-added-large-files
      - id: detect-private-key
```

Step 6: the configuration, field by field

- `repos`: the list of repositories that provide hooks. Each one is a small project on GitHub that packages one or more checks
- `repo`: the URL of one of them; `rev`: the release (a git tag of that repository) to use, so every clone runs exactly the same version of each check
- `hooks`: the checks of that repository to run, each named by its `id`
- `args`: extra command-line arguments for the check (`--fix`: ruff also applies its safe fixes)
- `files`: a regular expression on the file paths; the check runs only on matching files (`^src/`: mypy checks the package only)
- `repo: local`: a hook that runs a command of the project itself, here `entry: uv run --locked mypy`. `language: unsupported` tells pre-commit not to build an environment for it; `pass_filenames: false` runs mypy once, on the files that `pyproject.toml` names

The four checks of `pre-commit-hooks`: `end-of-file-fixer` (each file ends with exactly one newline), `trailing-whitespace` (no spaces at the end of a line), `check-added-large-files` (refuses files larger than 500 kB) and `detect-private-key` (refuses a private key: a secret pushed once stays in the history and must be revoked).

Step 6: install and run the hooks

Install the hooks once per clone, run them on every file once, then commit the configuration:

```
uv run pre-commit install          # once per clone: writes .git/hooks/pre-commit
uv run pre-commit run --all-files # every hook, on every file (on my code: all "Passed")
git add .pre-commit-config.yaml
git commit -m "chore: run ruff, mypy and file checks in a pre-commit hook"
```

If a hook changes one of your files (it prints "Failed" and "files were modified by this hook"), read the change with `git diff` and add that file too. The first run downloads the hooks from GitHub and prepares them: it takes a minute.

pre-commit builds a separate environment for each remote hook, which contains only that tool. That is fine for ruff, but mypy needs all the libraries that `heron` imports. So mypy is a local hook running in `.venv`.

The gate in action

From now on, every `git commit` runs the hooks on the changed files and is **blocked** if one fails. Try it with a badly formatted line, then throw the experiment away:

```
echo "x=1" >> src/heron/cli.py
git commit -am "test: a badly formatted line"   # ruff-format fails and fixes the file: no commit
git diff                                        # the hook rewrote the line as "x = 1"
git restore --staged --worktree src/heron/cli.py
```

When a hook **fixes** a file (the formatter, the end-of-file and whitespace hooks), the commit stops so that you can look at the change. If the fix is right, add the file again and repeat the same `git commit`. When a hook only **reports** a problem (mypy), fix the code yourself first.

The hooks run on your machine, so anyone can skip them with `git commit --no-verify`: this gate helps you, but it forces nobody. The gates that nobody can skip run on the server CI: it repeats the same checks, and a rule of the repository blocks every change that fails them.

Step 7: the README for uv

The README still says `PYTHONPATH=src python3 -m heron data/samples`. Update it **in your own words**: install with `uv sync`, run with `uv run heron data/samples`, and a status line that lists today's tooling (packaged, formatted and linted with ruff, typed with mypy strict, a pre-commit hook, an API reference built with Sphinx).

```
git add README.md
git commit -m "docs: describe the uv workflow in the README"   # the hooks run: all "Passed"
git push
```

The commit contains the README only. If a hook fixes a trailing space in your README, add the file again and repeat the commit.

Takeaways

- docstrings (PEP 257, NumPy style) state the contract; Sphinx builds the reference from them
- move checks **left**: tools catch mechanical faults at commit time, people review design
- **ruff** formats (no more style debates) and lints (likely bugs, security patterns); commit configuration, tool output and fixes separately
- every dependency is part of your **supply chain**: lock it, check new names, scan it
- **mypy strict** turns type hints into checked contracts; `X | None` plus narrowing removes a whole class of crashes
- **pre-commit** makes the checks automatic before every commit; CI repeats them for everyone

Appendix: every step in full

How to read the appendix

The slides of the lecture show the key lines of each step. The next slides show the **complete change** of every step, as `git diff` prints it, so that you can check your files line by line, or type a step you missed.

- a line that starts with `-` is removed, a line that starts with `+` is added, a line that starts with a space is unchanged and only shows where the change goes
- `@@ -23 +23,11 @@` gives the position of the lines that follow: line 23 of the old file becomes line 23 of the new file (11 lines); `-0,0` means a new file
- files and changes that a tool writes (`sphinx-apidoc` in step 2, `ruff format` in step 4) are not shown: the slide says which command makes them
- the descriptions in the docstrings (step 1), `docs/index.rst` (step 2) and `README.md` (step 7) are in your own words: the appendix shows mine, as an example
- every new file is written from scratch: nothing is copied from another file

Step 1 in full (1/2)

Commit `docs: NumPy-style docstrings for the public functions`: three files change, only in their docstrings. First `src/heron/parser.py`:

```
@@ -20,3 +20,16 @@ def _domain(header_value: str) -> str | None:
 def parse_eml(raw: str, email_id: str) -> ParsedEmail:
-    """Read one raw message once and keep only what the checks need."""
+    """Read one raw message once and keep only what the checks need.
+
+    Parameters
+    ----------
+    raw : str
+        The whole message, headers and body, as read from an ``.eml`` file.
+    email_id : str
+        The name of the message in the results, such as its file name.
+
+    Returns
+    -------
+    ParsedEmail
+        The parts of the message that the indicator checks read.
+    """
     msg = message_from_string(raw, policy=policy.default)
```

Step 1 in full (2/2)

Step 1, continued: `src/heron/scorer.py`, then `src/heron/cli.py`.

```
@@ -10,3 +10,14 @@ SUSPICIOUS_THRESHOLD = 3
 def triage(email: ParsedEmail) -> TriageResult:
-    """Run every check on one message and decide its verdict."""
+    """Run every check on one message and decide its verdict.
+
+    Parameters
+    ----------
+    email : ParsedEmail
+        The message, as returned by ``parse_eml``.
+
+    Returns
+    -------
+    TriageResult
+        The score of the message and its verdict.
+    """
     score = sum(check(email) for check in ALL_CHECKS)
```

```
@@ -33,3 +33,3 @@ def check_mail(path: Path) -> list[TriageResult]:
 def report(results: list[TriageResult]) -> None:
-    """A line for each email, with its verdict and its score."""
+    """Print one line for each email, with its verdict and its score."""
     for r in results:
```

Step 2 in full (1/2)

Commit `docs: generate an API reference with Sphinx`: two new files in `docs/`, written from scratch, and the two pages that `uv run sphinx-apidoc -o docs src/heron` writes (`docs/modules.rst` and `docs/heron.rst`, not shown). First `docs/conf.py`:

```
@@ -0,0 +1,10 @@
+"""Sphinx configuration: build the heron docs from the docstrings."""
+
+project = "heron"
+extensions = [
+    "sphinx.ext.autodoc",  # read the docstrings of the modules
+    "sphinx.ext.napoleon",  # understand the NumPy-style sections
+]
+napoleon_google_docstring = False  # this project writes NumPy style only
+html_theme = "sphinx_rtd_theme"
+exclude_patterns = ["_build"]
```

Step 2 in full (2/2)

Step 2, continued: my `docs/index.rst`. Yours says the same in your own words. The course repository also replaces `site/` with `docs/_build/` in its own `.gitignore`; yours, GitHub's Python template, already lists `docs/_build/`.

```
@@ -0,0 +1,14 @@
+heron
+=====
+
+Phishing email triage: heron reads ``.eml`` files and gives each message a
+verdict (benign, suspicious or phishing) and a score.
+
+.. code-block:: bash
+
+   uv sync                          # install heron and the dev tools from uv.lock
+   uv run heron data/samples        # one verdict per message
+
+.. toctree::
+
+   modules
```

Step 3 in full

Commit `chore: configure ruff (format and lint)`: the change to `pyproject.toml`.

```
@@ -23 +23,11 @@ build-backend = "hatchling.build"
 packages = ["src/heron"]
+
+[tool.ruff]
+line-length = 100
+src = ["src"]
+
+[tool.ruff.lint]
+select = ["E", "F", "I", "UP", "B", "SIM", "C4", "S"]  # S = security rules (bandit)
+
+[tool.ruff.lint.per-file-ignores]
+"tests/*" = ["E501", "S101"]  # pytest uses plain assert
```

Step 4 and step 5 in full

Commit `style: format the code with ruff`: nothing to type. `uv run ruff format .` makes the change (in `src/heron/features.py`: the `KEYWORDS` list, one item per line, and the `sum()` of `brand_score()` on one line); add `src`.

Commit `chore: type-check the package with mypy in strict mode`: the change to `pyproject.toml`.

```
@@ -33 +33,5 @@ select = ["E", "F", "I", "UP", "B", "SIM", "C4", "S"]  # S = security rules (ban
 "tests/*" = ["E501", "S101"]  # pytest uses plain assert
+
+[tool.mypy]
+strict = true
+files = ["src"]
```

Step 6 in full

Commit `chore: run ruff, mypy and file checks in a pre-commit hook`: `.pre-commit-config.yaml` is a new file, written from scratch.

```
@@ -0,0 +1,22 @@
+repos:
+  - repo: https://github.com/astral-sh/ruff-pre-commit
+    rev: v0.16.10
+    hooks:
+      - id: ruff-check
+        args: [--fix]
+      - id: ruff-format
+  - repo: local
+    hooks:
+      - id: mypy
+        name: mypy
+        entry: uv run --locked mypy  # the mypy of the project's .venv, with every dependency
+        language: unsupported  # pre-commit builds no environment: the project has one
+        files: ^src/
+        pass_filenames: false  # mypy checks files = ["src"] of pyproject.toml
+  - repo: https://github.com/pre-commit/pre-commit-hooks
+    rev: v6.0.0
+    hooks:
+      - id: end-of-file-fixer
+      - id: trailing-whitespace
+      - id: check-added-large-files
+      - id: detect-private-key
```

Step 7 in full

Commit `docs: describe the uv workflow in the README`: my change to `README.md` (only the changed lines). Yours says the same in your own words.

```
@@ -13 +13 @@ verdict: benign, suspicious, or phishing.
-The code is a package under `src/heron/`. From the repository root:
+Install heron and the dev tools with uv, then run it from the repository root:
@@ -16,2 +16,2 @@ The code is a package under `src/heron/`. From the repository root:
-PYTHONPATH=src python -m heron data/samples
+uv sync                          # create .venv from uv.lock (heron + dev tools)
+uv run heron data/samples
@@ -25,3 +26,4 @@ lists the options.
-Modular code: a package under `src/heron/` with one module per
-responsibility (models, parser, features, scorer, cli). Not packaged yet:
-there is no `pyproject.toml` and nothing to install.
+Packaged (`pyproject.toml`, src layout, committed `uv.lock`), formatted and
+linted (ruff), typed (mypy strict), with a pre-commit hook and an API
+reference built from the docstrings by Sphinx
+(`uv run sphinx-build -W docs docs/_build`).
```